Creative Software Design
7. Lab 2
STL – Standard Template Library

Dept. of Computer Science

Hanyang University

Finding Min/Max Values Using STL

- Check the arguments used in STL functions
  - A custom compare function can be used on the third parameter

```
#include <iostream>
#include <algorithm>
#include <list>
#include <vector>

int main(void) {
  int a = 10, b = 3;
  std::cout << "Min: " << std::min(a, b)
    << ", Max: " << std::max(a, b) << std::endl;
  std::cout << "Min: " << std::min({10, 3, 8, 1, 2})
    << ", Max: " << std::max({10, 3, 8, 1, 2}) << std::endl;

  std::vector<int> vec = {10, 3, 8, 1, 2};
  std::list<int> list = {10, 3, 8, 1, 2};
  std::cout << "Min: " << *std::min_element(vec.begin(), vec.end())
    << ", Max: " << *std::max_element(vec.begin(), vec.end()) << std::endl;
  std::cout << "Min: " << *std::min_element(list.begin(), list.end())
    << ", Max: " << *std::max_element(list.begin(), list.end()) << std::endl;

  return 0;
}
```

How to Sort STL Containers

- Different `sort()` are available for vector and list
  - `std::sort` for `std::vector`
  - Member function `sort()` for `std::list`
  - A custom compare function can be used as well

```
#include <iostream>
#include <algorithm>
#include <list>
#include <vector>

int main(void) {
  std::vector<int> vec = {10, 3, 8, 1, 2};
  std::list<int> list = {10, 3, 8, 1, 2};

  std::sort(vec.begin(), vec.end());
  list.sort();
  return 0;
}
```

Custom Compare Function

- Custom compare functions are required
  - When listing data in descending order
  - When specifying sorting criteria for non-number data

```
#include <iostream>
#include <algorithm>
#include <vector>

int cmp(const int& a, const int& b) {
   return a > b; // descending order
}

int main(void) {
  std::vector<int> vec = {10, 3, 8, 1, 2};

  std::sort(vec.begin(), vec.end(), cmp);

  return 0;
}
```

std::string

- C++ strings can be defined either using
  - C-style char arrays
  - `std::string` class
- Provides a set of useful member functions and STL iterators
- `std::string` automatically resize to the length of input

```
std::string str;

std::cin >> str;                // read a word (separated by a space,
                                // tab, enter)
std::getline(cin, str);         // read characters until the default
                                // delimiter '\n' is found
std::getline(cin, str, ':');    // read characters until the delimiter
                                // ':' is found
```

Two options of `getline()`

- `std::getline()` function

```
std::istream&
std::getline(std::istream& input, string str, char del);
```

- `std::cin.getline()` member function (or method)

```
std::istream&
std::istream.getline(char* s, int size, char del);
```

`std::set` & `std::map`

- Both are associative container classes
  - Manages stored data and its membership
- But stored data type are different
  - `std::set` stores only keys
  - `std::map` stores key and value paris
- Both Provide ***iterators*** to traverse keys in sequence
  - But traverse order may be different from insertion order

#1 – `sort()` in Descending Order

- Reuse example05.cpp of week07_1
- Complete `int comp(const T& a, const T& b);`
  - `a: left value`
  - `b: right value`
  - `return: a > b or a < b`

```
#include <iostream>

int main(void) {
  std::vector<int> numbers = {10, 3, 8, 1, 2};

  // print numbers in their original order
  // print numbers in ascending order
  // print numbers in descending order

  return 0;
}
```

#2 – `sort()` in Descending Order of GPA

- Reuse example03.cpp of week07_1
  - cp ../week07_1/example03.cpp ./example02.cpp
- Complete `int comp(const T& a, const T& b);`

```
#include <iostream>

int main(void) {
  /....../

    case 'p':
      std::sort(students.begin(), students.end(), comp);
      for (it = students.begin(); it != students.end(); it++) {
        it->Print();
      }
      break;



  /....../
  return 0;
}
```

#3 – EchoBot

- Reuse example03.cpp of week03_1

```
$ cp ../week03_1/example03.cpp ./
```

- Compare both options for EchoBot
  - `std::getline()`
  - `std::cin.getline()`

```
std::istream&
std::getline(std::istream& input, string str, char del);
```

```
std::istream&
std::istream.getline(char* s, int size, char del);
```

#4 – Duplicated ID Check-up

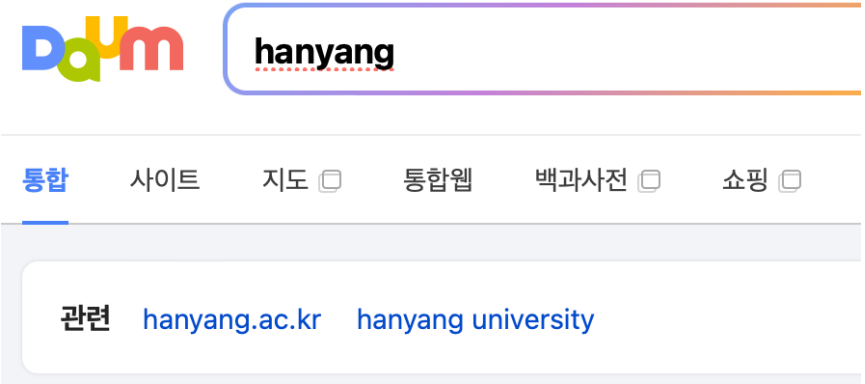
- Improve the previous program

```
$ cp example02.cpp example04.cpp
```

- Another ID if the entered ID already exists
  - How can we check if the current input ID is a duplicate?

#5 – Recommendation Systems

- Complete a simple recommendation system
  - Take a keyword from user input and display it
  - If the keyword need correction, display the corrected one
    - hanyang ➔ hanyang.ac.kr, hanyang univ
    - univ ➔ univ2026, univ meaning, unv, univ 2025, univ 2024
    - computr ➔ computer
  - Enter "quit" to terminate the program




```
computer
Showing results for computer
computr
Showing results for computr
 *Related: computer
```

All Done!!

- Let's submit to ***gitlab***
- Push to ***gitlab*** after each lab session
  - At your VM instance (and make sure **pwd** at "week07_2")

```
# You are at {Your local project directory}/week07_2
$ git add Makefile *.cpp *.h
$ git commit –m "submission of week07_2"
$ git push origin main
```